

第 8 章：反向传播

第 2 节：自动微分法 (Automatic Differentiation)

《深度学习基础》课程小组

2026 年 10 月 7 日

目录

- 1 四种求导方法
- 2 自动微分
- 3 前向模式自动微分
- 4 反向模式自动微分
- 5 总结
- 6 参考文献

1. 评估梯度的四种方法

- 利用梯度信息高效训练神经网络至关重要。
- 本质上，有**四种方法**评估神经网络误差函数的梯度：
 - ① 手动推导反向传播方程并显式实现
 - ② 数值微分（有限差分）
 - ③ 符号微分
 - ④ **自动微分**（automatic differentiation）

1.1 方法一：手动推导反向传播

- 多年来一直是神经网络的中流砥柱。
- 如果操作得当：代码高效，结果精确到数值精度。
- 缺点：
 - 推导和编码都耗时且易出错
 - 前向传播与反向传播方程分开编写，导致代码冗余
 - 模型更改时，前向和后向实现必须同步修改
 - 限制了快速探索不同架构的能力

1.2 方法二：数值微分

- 只需实现前向传播方程。
- 优点：实现简单，可用于调试其他方法。
- 缺点：
 - 计算精度有限（但对 SGD 影响不大）
 - 主要问题：可扩展性差， $\mathcal{O}(W^2)$
 - 每个参数需单独扰动，对现代大网络不可行

用途

将反向传播或自动微分的导数与中心差分结果比较，验证软件正确性。

1.3 方法三：符号微分

- 用专用软件自动解析推导过程（计算机代数）。
- 自动应用微积分规则（如链式法则）。
- **优点**：避免人为错误，机器精度，无扩展性问题。
- **缺点**：**表达式膨胀**（expression swell）。

表达式膨胀示例

$$f(x) = u(x)v(x) \tag{8.42}$$

$$f'(x) = u'(x)v(x) + u(x)v'(x) \tag{8.43}$$

$u(x)$ 和 $v(x)$ 在 f 和 f' 中都需要计算——**冗余**。
嵌套因子导致表达式复杂度**指数级增长**。

1.4 符号微分的表达式膨胀示例

两层网络示例 (soft ReLU 激活):

$$z = h(w_1x + b_1) \quad (8.44)$$

$$y = h(w_2z + b_2) \quad (8.45)$$

符号微分求得的导数:

$$\frac{\partial y}{\partial w_1} = \frac{w_2 x \exp(w_1 x + b_1 + b_2 + w_2 \ln[1 + e^{w_1 x + b_1}])}{(1 + e^{w_1 x + b_1})(1 + \exp(b_2 + w_2 \ln[1 + e^{w_1 x + b_1}]))} \quad (8.48)$$

- 复杂度显著高于原函数。
- $w_1x + b_1$ 等表达式多次重复。

1.5 符号微分的局限性：封闭形式与控制流

- 要求表达式以**封闭形式**给出。
- 无法处理**控制流操作**：
 - 循环、递归
 - 条件执行
 - 过程调用
- 这些是定义网络函数时**有价值**的构造。

关键限制

符号微分对**静态表达式树**进行代数变换，无法处理运行时才能确定结构的计算逻辑。

2. 方法四：自动微分

- 也称 autodiff 或**算法微分**。
- 目标：**不是**找导数的数学表达式，而是让计算机**自动生成梯度计算代码**。
- 仅需前向传播代码，即可生成梯度计算代码。
- **优点：**
 - 机器精度（同符号微分）
 - 利用**中间变量**避免冗余计算（比符号微分高效）
 - 能处理**控制流**（分支、循环、递归、过程调用）
- 自动微分是成熟领域，主要在**机器学习社区之外**发展。

2.1 自动微分的核心思想

- 取计算函数的代码（如神经网络的前向传播方程）。
- 在执行过程中**增加额外变量**来累积数值，从而获得导数。

两种主要形式

- **前向模式** (forward mode): 概念上更简单
- **反向模式** (reverse mode): 深度学习的主流选择
- 核心机制：将复杂函数分解为**基本运算**，用链式法则组合导数。
- 就像把复杂数学题拆解成基础运算，再综合结果。

2.2 前向模式 vs 反向模式

特性	前向模式	反向模式
计算方向	输入 $\rightarrow$ 输出	输出 $\rightarrow$ 输入
工作原理	计算中间变量同时算其对输入的导数	先前向记录计算图，再从输出反向计算梯度
效率特点	一次得一个输入的导数	一次得所有输入的导数
内存消耗	较低	较高（需保存中间结果）
适用场景	输入维度小、输出维度大	输入维度远大于输出维度（深度学习）

- 深度学习：参数百万级，损失函数标量输出 $\Rightarrow$ 反向模式是主流。

2.3 实现机制：计算图

- 框架执行张量运算时**动态构建有向无环图** (DAG)。
- **叶子节点**：输入数据或模型参数。
- **中间节点**：基本运算（加法、矩阵乘法等）。
- **根节点**：最终输出（如损失值）。

前向传播与反向传播

- **前向传播**：沿计算图从输入到输出，计算每个节点值和最终损失。
- **反向传播**：从根节点开始，沿反方向用链式法则将梯度传回每个节点。

现代框架通常存储**向量-雅可比乘积** (VJP)，而非完整雅可比矩阵。

3. 前向模式自动微分 (8.2.1)

- 为每个中间变量 z_i (主变量, primal) 附加一个切线变量 $\dot{z}_i$ 。
- 切线变量及其代码由软件环境自动生成。
- 代码传播元组 $(z_i, \dot{z}_i)$, 变量与导数并行计算。
- 原始函数由基本算子定义 (算术、取反、指数、对数、三角函数)。
- 这些算子的导数公式简单, 结合链式法则自动构建梯度计算代码。

关键点

一次前向传播只能计算对一个输入变量的导数。
若有 D 个输入, 需 D 次前向传播。

3.1 前向模式示例

考虑函数：

$$f(x_1, x_2) = x_1 x_2 + \exp(x_1 x_2) - \sin(x_2) \quad (8.49)$$

主变量（求值轨迹）：

$$v_1 = x_1 \quad (8.50)$$

$$v_2 = x_2 \quad (8.51)$$

$$v_3 = v_1 v_2 \quad (8.52)$$

$$v_4 = \sin(v_2) \quad (8.53)$$

$$v_5 = \exp(v_3) \quad (8.54)$$

$$v_6 = v_3 - v_4 \quad (8.55)$$

$$v_7 = v_5 + v_6 \quad (8.56)$$

3.2 切线变量的计算 I

求 $\partial f / \partial x_1$, 定义 $\dot{v}_i = \partial v_i / \partial x_1$ 。链式法则:

$$\dot{v}_i = \frac{\partial v_i}{\partial x_1} = \sum_{j \in \text{pa}(i)} \dot{v}_j \frac{\partial v_i}{\partial v_j} \quad (8.57)$$

其中 $\text{pa}(i)$ 是节点 i 的父节点集合。

3.2 切线变量的计算 II

切线变量方程：

$$\dot{v}_1 = 1 \quad (8.58)$$

$$\dot{v}_2 = 0 \quad (8.59)$$

$$\dot{v}_3 = \dot{v}_1 v_2 + v_1 \dot{v}_2 \quad (8.60)$$

$$\dot{v}_4 = \dot{v}_2 \cos(v_2) \quad (8.61)$$

$$\dot{v}_5 = \dot{v}_3 \exp(v_3) \quad (8.62)$$

$$\dot{v}_6 = \dot{v}_3 - \dot{v}_4 \quad (8.63)$$

$$\dot{v}_7 = \dot{v}_5 + \dot{v}_6 \quad (8.64)$$

初始条件： $\dot{v}_1 = 1, \dot{v}_2 = 0$ (对 x_1 求导)。

3.3 多输出与前向模式的雅可比矩阵

考虑两个输出 f_1, f_2 :

$$f_2(x_1, x_2) = (x_1 x_2 - \sin(x_2)) \exp(x_1 x_2) \quad (8.65)$$

- $\partial f_1 / \partial x_1$ 和 $\partial f_2 / \partial x_1$ 可在同一次前向传播中一起计算。
- 但若求对 x_2 的导数，需单独再跑一次前向传播。

雅可比矩阵

D 个输入、 K 个输出的函数:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_D} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_K}{\partial x_1} & \cdots & \frac{\partial f_K}{\partial x_D} \end{bmatrix} \quad (8.66)$$

一次前向传播产生雅可比矩阵的一列。计算完整雅可比矩阵需 D 次前向传播。

3.4 前向模式的局限性与反向模式的引入

- 前向模式： D 次前向传播计算完整雅可比矩阵。
- 适合输入少、输出多的网络 ($K \gg D$)。
- **但深度学习**：一个误差函数，大量参数（百万到数十亿）。
- 此时前向模式极其低效。
- 因此转向反向模式：基于导数数据在求值轨迹图中的反向流动。

关键对比

前向模式代价正比于输入维度 D 。

反向模式代价正比于输出维度 K 。

神经网络： $K = 1$ （标量损失）， D 百万级 $\Rightarrow$ 反向模式获得 $O(D)$ 倍加速。

4. 反向模式自动微分 (8.2.2)

- 可视为误差反向传播的推广。
- 为每个中间变量 v_i 附加伴随变量 $\bar{v}_i$:

$$\bar{v}_i = \frac{\partial f}{\partial v_i} \quad (8.68)$$

- 从输出开始，反向使用链式法则：

$$\bar{v}_i = \sum_{j \in \text{ch}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i} \quad (8.69)$$

其中 $\text{ch}(i)$ 是节点 i 的子节点集合。

- 伴随变量的连续求值代表信息在图中反向流动。

4.1 反向模式示例

对函数 (8.49) 的伴随变量方程：

$$\bar{v}_7 = 1 \quad (8.70)$$

$$\bar{v}_6 = \bar{v}_7 \quad (8.71)$$

$$\bar{v}_5 = \bar{v}_7 \quad (8.72)$$

$$\bar{v}_4 = -\bar{v}_6 \quad (8.73)$$

$$\bar{v}_3 = \bar{v}_5 v_5 + \bar{v}_6 \quad (8.74)$$

$$\bar{v}_2 = \bar{v}_3 v_1 + \bar{v}_4 \cos(v_2) \quad (8.75)$$

$$\bar{v}_1 = \bar{v}_3 v_2 \quad (8.76)$$

- 从输出开始，**反向流动**到输入。
- 即使有多个输入，只需**一次反向传播**。
- 对神经网络， E 对权重和偏置的导数即为相应伴随变量。
- 若有多个输出，需对每个输出单独反向传播。

4.2 反向模式的内存与实现考量

- 反向模式通常比前向模式**更耗内存**。
- 原因：所有**中间主变量**必须存储，供反向传播使用。
- 前向模式：主变量和切线变量一起计算，用完即可丢弃。
- 因此前向模式通常**更容易实现**。

计算开销

两种模式的单次传播均不超过函数求值计算代价的**6 倍**。
实践中通常为**2-3 倍** (Griewank and Walther, 2008)。

4.3 混合模式：Hessian-向量积

- 前向与反向模式的混合也很有用。
- 典型应用：Hessian 矩阵与向量的乘积，无需显式构造完整 Hessian。

Hessian-向量积的计算步骤

- ① 设向量 $\mathbf{v}$ 和点 $\mathbf{x}$ 。
- ② 前向模式：令 $\dot{\mathbf{x}} = \mathbf{v}$ ，得到方向导数 $\mathbf{v}^\top \nabla f$ 。
- ③ 反向模式：对该结果微分，得到 $\nabla^2 f \mathbf{v} = \mathbf{H} \mathbf{v}$ 。

- 若参数数量为 W ，HVP 复杂度为 $\mathcal{O}(W)$ 。
- 即使 Hessian 本身大小为 $W \times W$ 。
- 显式计算完整 Hessian： $\mathcal{O}(W^2)$ 。

5. 本节总结

- 四种求导方法：
 - 手动反向传播：高效但耗时易错
 - 数值微分：简单但 $\mathcal{O}(W^2)$
 - 符号微分：机器精度但表达式膨胀，不支持控制流
 - 自动微分：机器精度、高效、支持控制流
- 自动微分核心思想：增强代码，通过额外变量累积导数。
- 前向模式：
 - 传播元组 $(z_i, \dot{z}_i)$ ，变量与导数并行计算
 - 一次传播产生雅可比矩阵一列
 - 适合 $K \gg D$ ，但深度学习不适用
- 反向模式：
 - 定义伴随变量 $\bar{v}_i = \partial f / \partial v_i$
 - 从输出反向传播，一次得所有输入的导数
 - 内存开销大，但深度学习主流选择
- 混合模式：Hessian-向量积， $\mathcal{O}(W)$ 复杂度。

6. 参考文献

- 教材: Bishop, C. M. & Bishop, H. (2023). Deep Learning Foundations and Concepts. Springer Nature Switzerland AG.
- Baydin, A. G. et al. (2018). Automatic differentiation in machine learning: a survey.
- Griewank, A. & Walther, A. (2008). Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation.
- Pearlmutter, B. A. (1994). Fast exact multiplication by the Hessian.
- Grosse, R. (2018). Automatic differentiation.